School of
Management and Law

MDM
Web Apps mit Java

Adrian Moser
adrian.moser@zhaw.ch, FS2026

AACSB ACCREDITED
EQUIS ACCREDITED
ASSOCIATION OF MBAS ACCREDITED

Inhalt

DJL DeepJavaLibrary
Spring Boot / Spring Web
REST Services / Controllers

DJL Ziele

Ziele

- Einfachere Übernahme von ML-Code im Produktion
- neuen Machine-Learning-Entwicklern Einstieg erleichtern
- modularer, wiederverwendbarer Code
- Code einmal schreiben und auf jeder Deep-Learning-Engine ausführen
- unabhängig von einer Deep-Learning-Engine

DeepJavaLibrary

DJL Engines

Idee

- unabhängig von einer Deep-Learning-Engine
- Jederzeit Wechsel möglich

Verfügbare Engines

- PyTorch
- TensorFlow
- MXNet
- ONNXRuntime
- … und weitere

Weitere Informationen:
https://docs.djl.ai/docs/engine.html

Java Grundlagen

Spring Boot

Java Grundlagen

Voraussetzungen

JDK: Java Development Kit (empfohlen LTS-Version)

Visual Studio Code mit Coding Pack for Java / Spring Boot Extension Pack

Spring Boot mit Spring Web

Java Framework für Web-Anwendungen

Maven

Java Dependency Management

Laden von Bibliotheken (z.B. DJL) aus Java

Apache Maven

Ideen

- Best Practices
- Grundsatz: **Convention over Configuration**
- Dependency Management

Dependency Management

- In der Praxis ist jede Software auf **Libraries** angewiesen
- Maven kann diese automatisch in der richtigen Version aus dem Internet laden
- Zudem kann Maven **transitive Dependencies** (Libraries, welche eine verwendete Library benötigt) automatisch ebenfalls laden

Apache Maven

POM

- Project Object Model
- Hierarchisch

Konventionen

- Standard-Projektstruktur

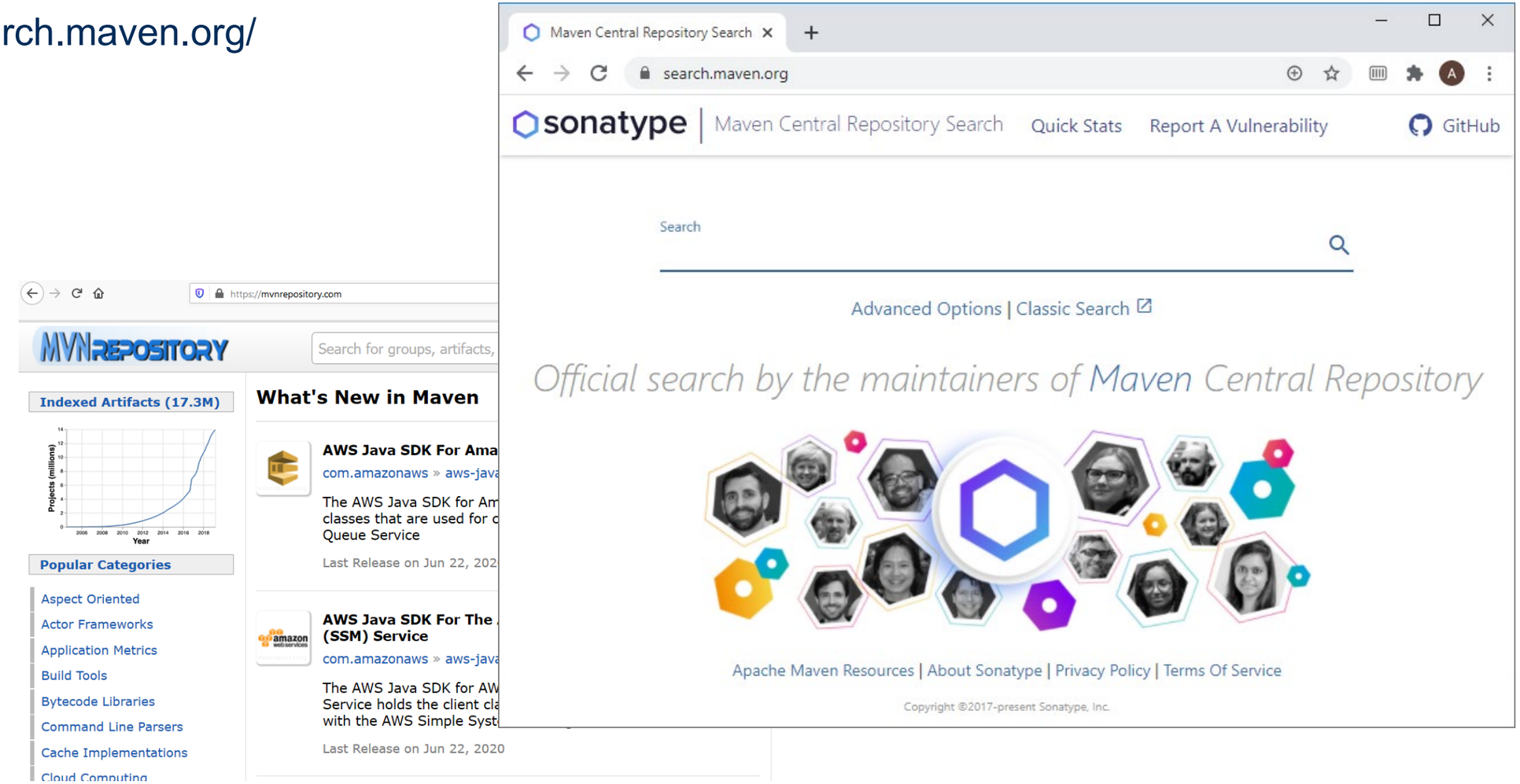
```
<project>
<modelVersion>4.0.0</modelVersion>

<groupId>com.mycompany.app</groupId>
<artifactId>my-module</artifactId>
<version>1</version>
</project>
```

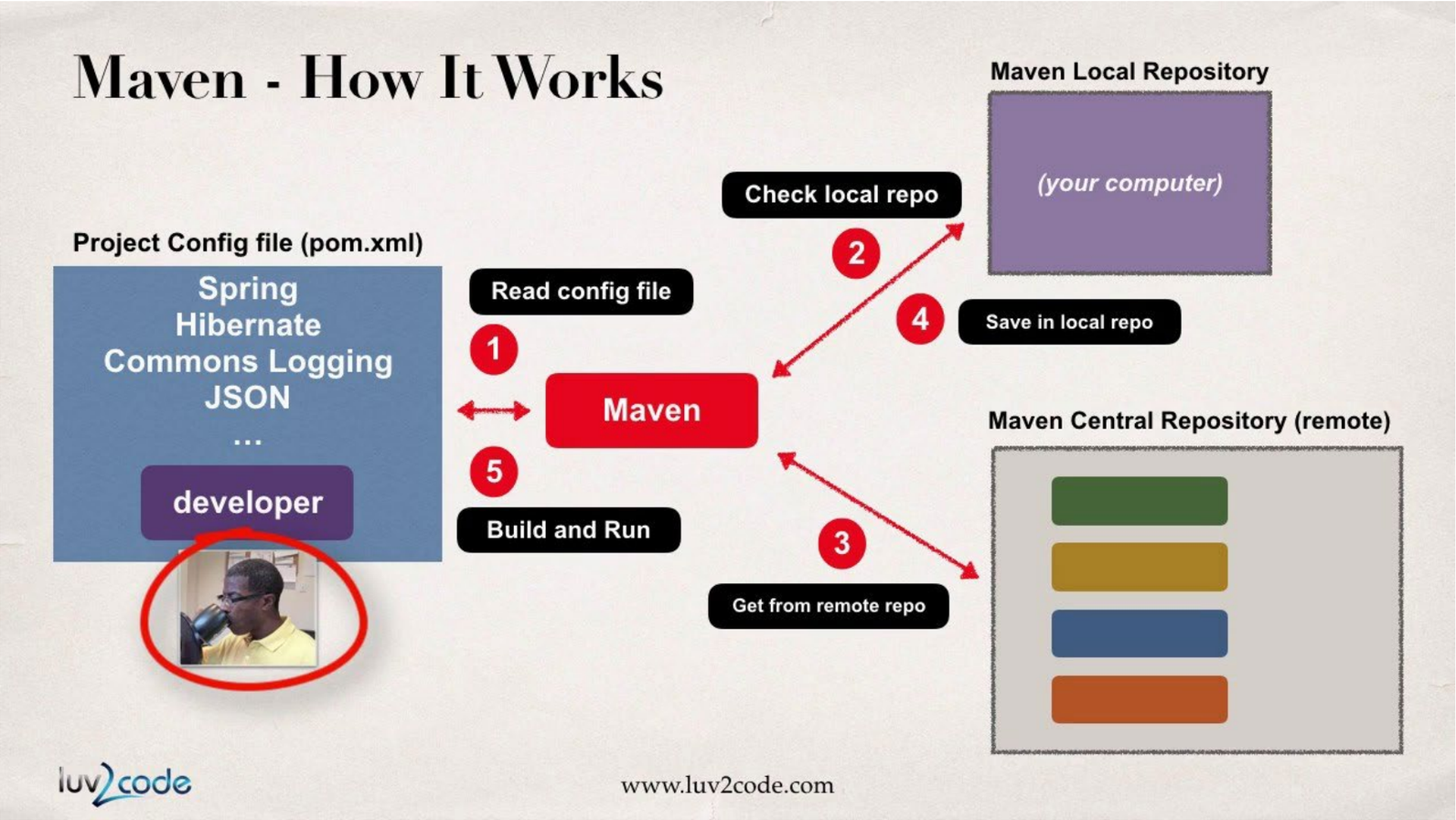
```
<Projektverzeichnis>
 src ..................................................Alle Eingabedateien
  main ...................... Eingabedateien für das eigentliche Produkt
   java ......................................Java-Quelltextdateien
   resources ......andere Dateien, die vom Projekt benötigt werden
  test ............... Eingabedateien, die für Testläufe benötigt werden
   java ............................Testfälle für automatisierte Tests
 target ........................Zielverzeichnis für alle erzeugten Dateien
  classes ................ Zielverzeichnis für kompilierte Java-Klassen
 pom.xml ........................................Konfigurationsdatei
```

Maven Repository / Maven Search

https://search.maven.org/



Maven Dependency Management



https://www.youtube.com/watch?v=m_eWc9pjyg4

Spring Web Projekt erstellen

Ziel: DJL in Spring Boot Web Projekt einbetten, um einen REST-Service azubieten

Cheat Sheet

Erstellung eines Spring Web Projektes in Visual Studio Code

1. **Neues Fenster in Visual Studio Code öffnen (Menu File)**
2. **Ctrl-Shift-P: Spring Initializr: Create a Maven Project…**
3. **Specify Spring Boot Version: 3.5.9 (oder ähnlich)**
4. **Specify Project Language: Java**
5. **Input Group Id: ch.zhaw.deeplearningjava**
6. **Input Artifact Id: playground / Enter**
7. **Specify Packaging Type: Jar**
8. **Specify Java Version: 25**
9. **Search for Dependencies: Spring Web**
10. **Verzeichnis erstellen, Generate into this folder**
11. **Info-Message in Visual Studio Code: Open**
12. **Yes, I trust the authors**

SpringBootApplication

Hauptklasse der Web-Application (main-Methode)

Spring wird mit SpringApplication.run gestartet

Achtung: Java-Projekt muss komplett geladen sein

```
@SpringBootApplication
public class PlaygroundApplication {

        public static void main(String[] args) {
                SpringApplication.run(PlaygroundApplication.class, args);
        }

}
```

Start mit Ctrl-F5 (Menu Run)

```
2022-11-28 13:08:20.177  ...: Tomcat started on port(s): 8080 (http) with context path ''
2022-11-28 13:08:20.190  ...: Started PlaygroundApplication in 3.141 seconds (JVM running for 4.119)
```

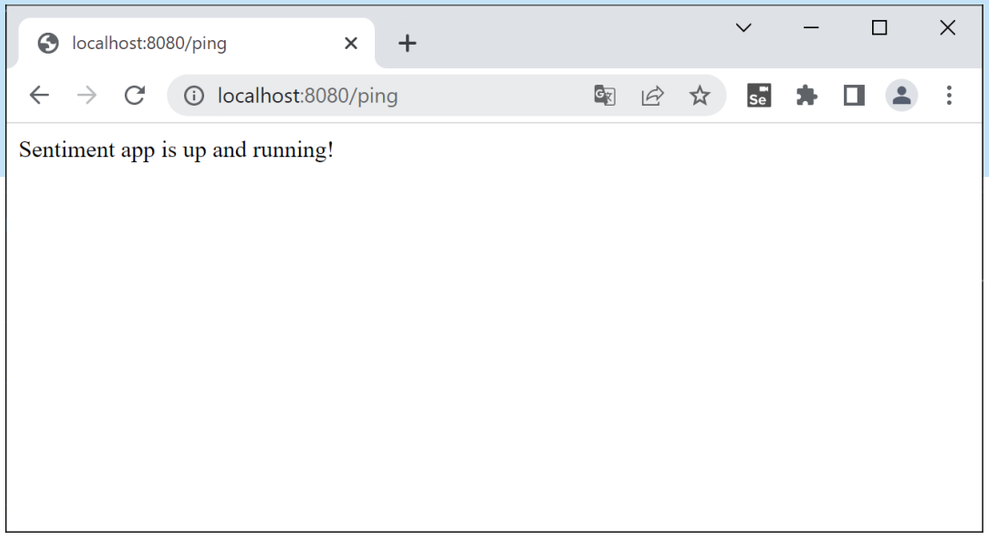
REST Controller

Microservice-Framework, analog zu Python Flask, allerdings auf Port 8080

```
@RestController
public class SentimentController {

    @GetMapping("/ping")
    public String ping() {
        return "Sentiment app is up and running!";
    }

    @GetMapping("/count")
    public int count() {
        return 42;
    }
}
```



DJL Installation

DJL zum Projekt hinzufügen

- Ctrl-Shift-P: Add a Maven Dependency
- **ai.djl.api und weitere Bibliotheken** auswählen
- Resultat im pom.xml überprüfen

```
<dependency>
        <groupId>ai.djl.pytorch</groupId>
        <artifactId>pytorch-model-zoo</artifactId>
        <version>0.36.0</version>
</dependency>
<dependency>
        <groupId>ai.djl</groupId>
        <artifactId>model-zoo</artifactId>
        <version>0.36.0</version>
</dependency>
<dependency>
        <groupId>ai.djl</groupId>
        <artifactId>basicdataset</artifactId>
        <version>0.36.0</version>
</dependency>
<dependency>
        <groupId>ai.djl</groupId>
        <artifactId>api</artifactId>
        <version>0.36.0</version>
</dependency>
```

DJL Beispiel

Sentiment Analysis

Initialisierung des Modells

Initialisierung (Übersicht)

Criteria: Enthält Kriterien, um ein Modell zu finden.

Danach wird das Modell geladen.

Aus dem Modell wird ein Predictor erstellt, der im REST-Service verwendet werden kann.

```
try {
        Criteria<String, Classifications> criteria = Criteria.builder()
                                .optApplication(Application.NLP.SENTIMENT_ANALYSIS)
                                .setTypes(String.class, Classifications.class)
                                // This model was traced on CPU and can only run on CPU
                                .optDevice(Device.cpu())
                                .optProgress(new ProgressBar())
                                .build();
        ZooModel<String, Classifications> model = criteria.loadModel();
        predictor = model.newPredictor();
} catch (Exception e) {
        logger.error("Cannot build Predictor", e);
}
```

Classifications

Der (vereinfachte) Output-Typ «Classifications»:

```
public class Classifications implements JsonSerializable {

    protected List<String> classNames;

    protected List<Double> probabilities;
}
```

- Eine Liste der Klassen-Namen
- Eine Liste der Wahrscheinlichkeiten (eine für jede Klasse)

Initialisierung (Modell)

Application: Enthält Applikationstyp (siehe folgende Seiten)

setTypes: Definiert Input und Output-Typen

```
Criteria<String, Classifications> criteria = Criteria.builder()
         .optApplication(Application.NLP.SENTIMENT_ANALYSIS)
         .setTypes(String.class, Classifications.class)
         // This model was traced on CPU and can only run on CPU
         .optDevice(Device.cpu())
         .optProgress(new ProgressBar())
         .build();
```

Builder Pattern

Beobachtung: Bei DJL werden sehr viele Aufrufe mit Punkt (.) verwendet.

Builder Pattern (Fluent API)
DeepJavaLearning verwendet das sogenannte BuilderPattern, um Parameter (benötigte und optionale) zu setzen. Dabei werden Konfigurationsmethoden über die **Punkt-Notation** aufgerufen.
Methode beginnt mit opt: Optional beginnt mit set: Required

DJL Application

Aktuell enhält Deep Java Library nebenstehende Applikationen.

Eine Beschreibung findet sich in der Klasse ai.djl.Application

```
CV.IMAGE_CLASSIFICATION
CV.OBJECT_DETECTION
CV.SEMANTIC_SEGMENTATION
CV.INSTANCE_SEGMENTATION
CV.POSE_ESTIMATION
CV.ACTION_RECOGNITION
CV.WORD_RECOGNITION
CV.IMAGE_GENERATION
CV.IMAGE_ENHANCEMENT
NLP.FILL_MASK
NLP.MACHINE_TRANSLATION
NLP.MULTIPLE_CHOICE
NLP.QUESTION_ANSWER
NLP.SENTIMENT_ANALYSIS
NLP.TEXT_CLASSIFICATION
NLP.TEXT_EMBEDDING
NLP.TOKEN_CLASSIFICATION
NLP.WORD_EMBEDDING
```

Sentiment-Modell verwenden

Verwendung in Spring REST-Service

Sentiment Service

Einmalige Erstellung des Objekts SentimentAnalysis (Controller ist Singleton)

Aufruf von «predict» für jeden Request

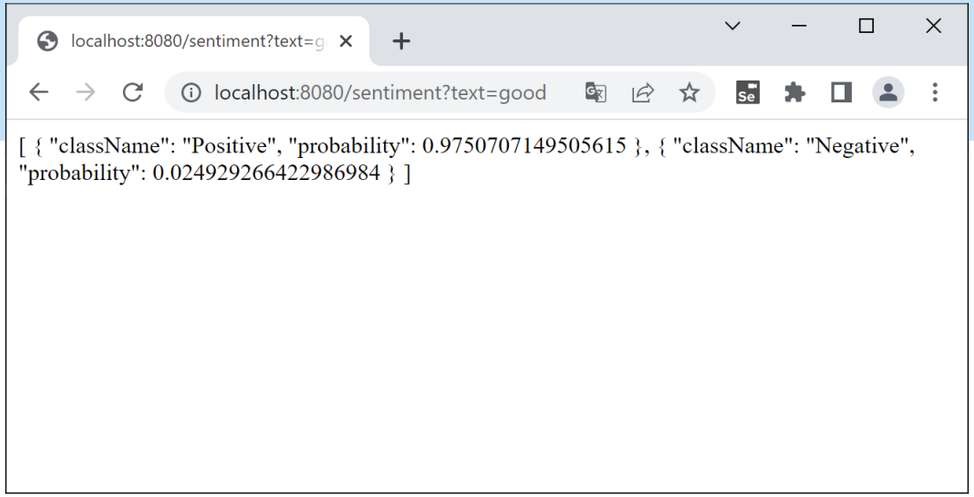
Rückgabe des Resultats als JSON

```
public class SentimentController {

    private SentimentAnalysis analysis = new SentimentAnalysis();

    @GetMapping("/sentiment")
    public String predict(@RequestParam(name="text", required = true) String text) throws Exception {
        var result = analysis.predict(text);
        return result.toJson();
    }

}
```



UI

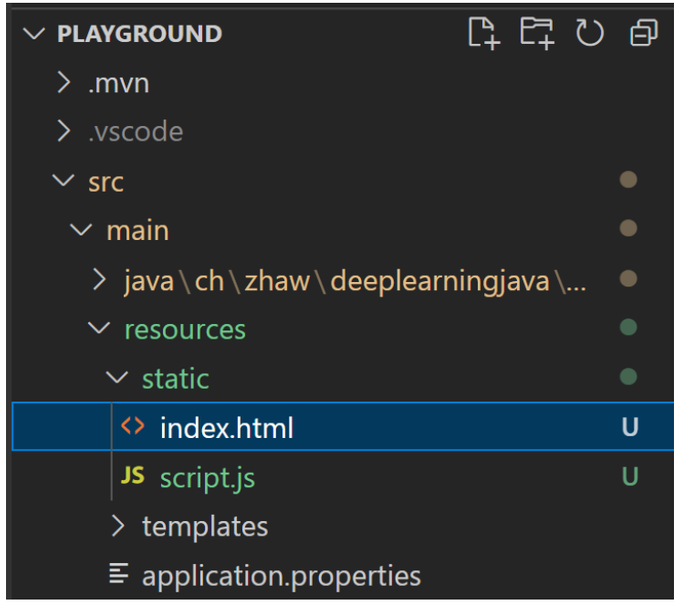
Ergänzung mit einfachem UI

Bereits bekannt aus Python-Beispiel

Static Resources in Spring Boot

src/main/resources/static

- Spring Boot zeigt Dateien aus diesem Verzeichnis automatisch an
- index.html als Einstiegsseite
- script.js für Aufruf REST-Service

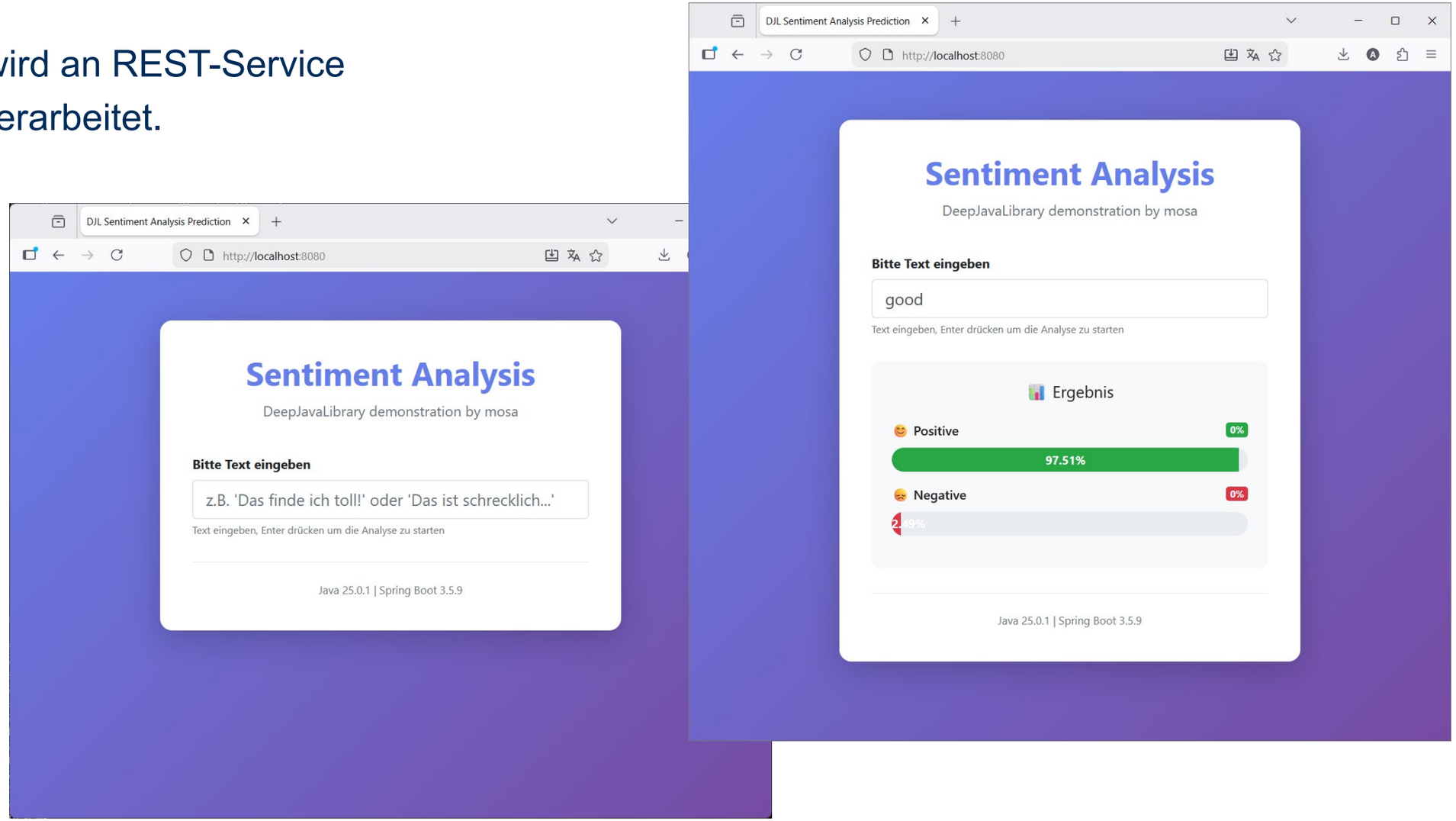


UI

Inhalt des Input-Feldes wird an REST-Service geschickt und von DJL verarbeitet.

Rückgabewert als JSON.

Grafische Anzeige im UI.



Weiterführende Informationen

Sentiment Analysis Example (Original)
https://github.com/deepjavalibrary/djl/blob/master/examples/src/main/java/ai/djl/examples/inference/nlp/SentimentAnalysis.java